IF25-22014 PENGEMBANGAN APLIKASI WEB

Pertemuan 11
Kerangka Kerja FastAPI

Membangun API JSON yang divalidasi otomatis dan dipanggil dari Next.js

Tim Pengajar
Tahun 2026. Rujukan: Dokumentasi resmi FastAPI (`fastapi.tiangolo.com`) dan Pydantic (`docs.pydantic.dev`).

ITERA

DIKTISAINTEK BERDAMPAK

Program Studi Teknik Informatika
Institut Teknologi Sumatera

PETA PERTEMUAN

Perjalanan kalian hari ini

ITERA

Fase	Menit	Yang kalian lakukan	Pertanyaan wajar	Bila tersendat
Pembuka	0–10	Menjawab tiga soal minggu lalu dengan jari	Masih ingat yang kemarin?	Catat nomor yang salah, cek di modul minggu lalu
Coba dulu	10–30	Mengerjakan tiga misi berpasangan	Boleh salah?	Tulis dugaan; salah itu bahan belajar
Materi	30–85	Mengikuti 8 bagian: tebak, ikuti bersama, engsel	Sedang di bagian mana?	Lihat penanda di atas judul atau klik Outline
Hands-on	85–120	Mengerjakan tingkat 1 sampai 3 dari `starter/`	Sudah benar belum?	Periksa berpasangan; buka modul bagian yang sama
Papan	120–140	Menunjukkan atau menanggapi hasil teman	Kenapa kode itu jalan?	Ajukan satu pertanyaan mengapa
Exit ticket	140–150	Menjawab dua soal tanpa catatan	Apa yang belum paham?	Tulis di kolom refleksi

Penanda fase dan bagian ada di atas setiap judul slide. Kotak **Ikuti bersama** berarti ketik di laptop kalian sekarang.

Soal dari Pertemuan 10

ITERA

Soal 1. `class Matkul(BaseModel)` adalah contoh?

- **A** dict
- **B** pewarisan
- **C** modul
- **D** comprehension

Soal 2. Dimana validasi sebaiknya dilakukan?

- **A** konstruktor atau setter
- **B** di luar class
- **C** di print
- **D** tidak perlu

Soal 3. Kode status untuk isian tidak valid (P1)?

- **A** 200
- **B** 302
- **C** 400 atau 422
- **D** 500

KUNCI

1B: mewarisi BaseModel. 2A: object yang lolos dijamin benar. 3C: salah di permintaan.

Halaman siap, datanya dari mana?

ITERA

Halaman KRS Next.js dari P7 sudah memanggil `http://localhost:8000/matkul`, tetapi alamat itu belum ada: browser menampilkan *Gagal memuat*.
Data mata kuliah juga harus diperiksa sebelum disimpan, karena validasi di browser bisa dilewati (P1).

PIKIRKAN

Program seperti apa yang harus berjalan di port 8000? Apa yang ia terima, dan apa yang ia kirim balik?

```
useEffect(() ⇒ {
  fetch("http://localhost:8000/matkul")
    .then((r) ⇒ r.json())
    .then(setData)
    .catch(setGalat);
}, []);
// TypeError: Failed to fetch
```

Misi kalian, 20 menit

ITERA

Berdua atau bertiga, pakai apa saja yang kalian tahu. Berkas awal ada di folder hands-on, `starter/`.

1 Server pertama

Di `starter/`, jalankan `fastapi dev main.py`. Buka `localhost:8000` dan `localhost:8000/docs`. Apa yang kalian lihat?

2 Satu alamat baru

Tambahkan alamat `/halo/{nama}` yang membalas `{"pesan": "Halo, nama"}`.

3 Kirim yang salah

Di halaman `/docs`, kirim mata kuliah dengan SKS berupa teks. Kode status berapa yang kembali?

Boleh

Bertanya ke teman, mencoba di browser, salah.

Belum dulu

Mencari di internet atau bertanya ke AI.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan (klik judul untuk melompat)
1	**Pengenalan FastAPI**	**Menjelaskan FastAPI dan alasan memilihnya untuk API**
2	Arsitektur FastAPI	Menjelaskan perjalanan request di dalam aplikasi FastAPI
3	Instalasi dan konfigurasi	Memasang, menjalankan, dan mengonfigurasi FastAPI termasuk CORS
4	Routing	Membuat path operation dengan path parameter, query parameter, dan APIRouter
5	Request dan response model	Memvalidasi body dengan model Pydantic dan mengatur response
6	Model dan integrasi basis data	Memisahkan model API dari penyimpanan dan menyiapkan dependency basis data
7	Keamanan dan autentikasi	Menjelaskan autentikasi dengan token dan praktik keamanan dasar API
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Kerangka kerja API Python

ITERA

FastAPI
Kerangka kerja web Python untuk membangun API yang memakai type hints untuk memvalidasi data masuk, mengubah data keluar menjadi JSON, dan membuat dokumentasi interaktif secara otomatis.

Pyramid di versi lama mata kuliah ini menyajikan halaman HTML dari server. Di stack baru, halaman dibangun Next.js; server cukup menyediakan data JSON. FastAPI dibuat khusus untuk itu, cepat karena berbasis ASGI dan mendukung `async`.

```
from fastapi import FastAPI

app = FastAPI(title="API KRS")

@app.get("/")
def beranda():
    return {"pesan": "API KRS berjalan"}
```

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengenalan FastAPI	Inti: FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	**Arsitektur FastAPI**	**Menjelaskan perjalanan request di dalam aplikasi FastAPI**
3	Instalasi dan konfigurasi	Memasang, menjalankan, dan mengonfigurasi FastAPI termasuk CORS
4	Routing	Membuat path operation dengan path parameter, query parameter, dan APIRouter
5	Request dan response model	Memvalidasi body dengan model Pydantic dan mengatur response
6	Model dan integrasi basis data	Memisahkan model API dari penyimpanan dan menyiapkan dependency basis data
7	Keamanan dan autentikasi	Menjelaskan autentikasi dengan token dan praktik keamanan dasar API
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Perjalanan satu request

ITERA

Pyramid memakai pola MVC dengan template. FastAPI berperan sebagai lapisan API saja:

1 Masuk

Middleware (misalnya CORS) memeriksa request, lalu path dan metode dicocokkan ke **path operation**.

2 Diperiksa

Parameter dan body divalidasi **model Pydantic**; dependency seperti sesi basis data disiapkan.

3 Dibalas

Fungsi mengembalikan object Python; FastAPI menyaring dengan **response model** lalu mengirim JSON.

Ingat

Di proyek: `main.py` membuat app, `routers/` berisi path operation per sumber daya, `schemas.py` model Pydantic, `models.py` tabel basis data (P12).

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengenalan FastAPI	Inti: FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	Inti: Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	**Instalasi dan konfigurasi**	**Memasang, menjalankan, dan mengonfigurasi FastAPI termasuk CORS**
4	Routing	Membuat path operation dengan path parameter, query parameter, dan APIRouter
5	Request dan response model	Memvalidasi body dengan model Pydantic dan mengatur response
6	Model dan integrasi basis data	Memisahkan model API dari penyimpanan dan menyiapkan dependency basis data
7	Keamanan dan autentikasi	Menjelaskan autentikasi dengan token dan praktik keamanan dasar API
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Pasang dan jalankan

ITERA

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install "fastapi[standard]"

fastapi dev main.py
# Serving at: http://127.0.0.1:8000
# API docs: http://127.0.0.1:8000/docs

# alternatif
uvicorn main:app --reload --port 8000
```

`uvicorn main:app`: menjalankan object `app` di berkas `main.py`; dipakai juga saat deployment.

CORS untuk Next.js

ITERA

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Inilah salah satu penyebab *Failed to fetch* di masalah minggu ini.

Ikuti bersama: server FastAPI pertama

ITERA

1. Aktifkan venv, lalu pasang FastAPI.
   `pip install "fastapi[standard]"`
2. Buat `main.py` berisi app dan endpoint `/` dari Bagian 1.
3. Jalankan server pengembangan.
   `fastapi dev main.py`
4. Buka dokumentasinya.
   `http://localhost:8000/docs`
5. Pilih endpoint GET, klik *Try it out*, lalu *Execute*.

YANG HARUS TERLIHAT

Terminal menulis alamat server dan docs. Halaman /docs menampilkan endpoint kalian, dan Execute memberi respons 200 berisi JSON.

Kalau berbeda

fastapi: command not found: venv belum aktif. *Address already in use*: ada server lain di port 8000; hentikan atau tambah `--port 8001`. Perubahan tidak terlihat: simpan berkasnya.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengenalan FastAPI	Inti: FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	Inti: Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	Instalasi dan konfigurasi	Inti: Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.
4	**Routing**	**Membuat path operation dengan path parameter, query parameter, dan APIRouter**
5	Request dan response model	Memvalidasi body dengan model Pydantic dan mengatur response
6	Model dan integrasi basis data	Memisahkan model API dari penyimpanan dan menyiapkan dependency basis data
7	Keamanan dan autentikasi	Menjelaskan autentikasi dengan token dan praktik keamanan dasar API
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Path dan query parameter

ITERA

```
@app.get("/matkul")
def daftar(sks: int | None = None, batas: int = 10):
    hasil = [m for m in DATA if sks is None or m["sks"] == sks]
    return hasil[:batas]

@app.get("/matkul/{kode}")
def detail(kode: str):
    for m in DATA:
        if m["kode"] == kode:
            return m
    raise HTTPException(status_code=404, detail="Mata kuliah
tidak ditemukan")

@app.get("/halo/{nama}")
def halo(nama: str):
    return {"pesan": f"Halo, {nama}"}
```

Jawaban misi 2. Untuk banyak sumber daya, kelompokkan dengan `APIRouter(prefix="/matkul")`.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengenalan FastAPI	**Inti:** FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	**Inti:** Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	Instalasi dan konfigurasi	**Inti:** Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.
4	Routing	**Inti:** Dekorator membuat endpoint; path parameter menunjuk, query parameter menyaring.
5	**Request dan response model**	**Memvalidasi body dengan model Pydantic dan mengatur response**
6	Model dan integrasi basis data	Memisahkan model API dari penyimpanan dan menyiapkan dependency basis data
7	Keamanan dan autentikasi	Menjelaskan autentikasi dengan token dan praktik keamanan dasar API
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Model untuk masuk dan keluar

```
from pydantic import BaseModel, Field

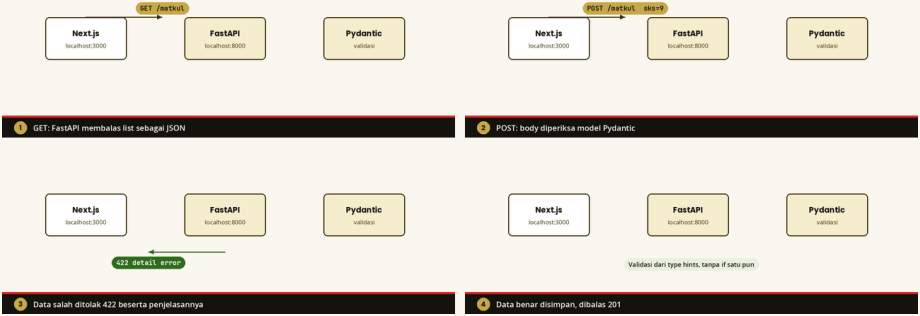
class MatkulIn(BaseModel):
    kode: str = Field(pattern=r"^IF\d{2}-\d{5}$")
    nama: str = Field(min_length=3)
    sks: int = Field(ge=1, le=6)

class MatkulOut(MatkulIn):
    id: int

@app.post("/matkul",              response_model=MatkulOut,
status_code=201)
def tambah(m: MatkulIn):
    baru = {"id": len(DATA) + 1, **m.model_dump()}
    DATA.append(baru)
    return baru
```

`status_code=201`: Created, kode yang tepat untuk data baru (P1).

Validasi dalam gerak



Satu endpoint menolak data salah dan menerima data benar tanpa satu pun if di kode kalian.

Kode status yang tepat

ITERA

Endpoint `PUT /matkul/{kode}` dipanggil untuk kode yang tidak ada. Balasan yang paling tepat?

- **A** 200 dengan `null`
- **B** 404 Not Found
- **C** 422 Unprocessable
- **D** 500 Server Error

JAWABAN: B

Sumber daya yang dituju tidak ada: 404. 422 untuk body yang tidak lolos validasi, 500 untuk kesalahan kode server.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengenalan FastAPI	**Inti:** FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	**Inti:** Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	Instalasi dan konfigurasi	**Inti:** Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.
4	Routing	**Inti:** Dekorator membuat endpoint; path parameter menunjuk, query parameter menyaring.
5	Request dan response model	**Inti:** Body divalidasi model Pydantic, ditolak 422 bila salah; response model menyaring keluaran.
6	**Model dan integrasi basis data**	**Memisahkan model API dari penyimpanan dan menyiapkan dependency basis data**
7	Keamanan dan autentikasi	Menjelaskan autentikasi dengan token dan praktik keamanan dasar API
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Dependency injection

```
from typing import Annotated
from fastapi import Depends

def get_db():
    db = SesiBasisData() # dibuat per request
    try:
        yield db
    finally:
        db.close()

DB = Annotated[Sesi, Depends(get_db)]

@app.get("/matkul")
def daftar(db: DB):
    return db.ambil_semua_matkul()
```

Pola yang sama dipakai untuk pengguna yang sedang login di Bagian 7. ORM SQLAlchemy dari slide lama dibahas penuh di P12.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengenalan FastAPI	**Inti:** FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	**Inti:** Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	Instalasi dan konfigurasi	**Inti:** Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.
4	Routing	**Inti:** Dekorator membuat endpoint; path parameter menunjuk, query parameter menyaring.
5	Request dan response model	**Inti:** Body divalidasi model Pydantic, ditolak 422 bila salah; response model menyaring keluaran.
6	Model dan integrasi basis data	**Inti:** Depends menyuntikkan sesi basis data ke setiap endpoint; tabelnya di P12.
7	**Keamanan dan autentikasi**	**Menjelaskan autentikasi dengan token dan praktik keamanan dasar API**
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Praktik keamanan dasar

ITERA

Ancaman atau kebutuhan	Praktiknya di FastAPI
password bocor	simpan hash, bukan teks asli
siapa yang memanggil?	login memberi token (JWT); request membawa header `Authorization`
khusus pengguna login	dependency pembaca token; 401 bila tidak sah
berhak atau tidak	periksa peran; 403 bila tidak berhak
data berbahaya	validasi Pydantic; query lewat ORM, bukan SQL rakitan
asal yang tidak dikenal	CORS hanya untuk asal frontend kalian

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengenalan FastAPI	**Inti:** FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	**Inti:** Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	Instalasi dan konfigurasi	**Inti:** Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.
4	Routing	**Inti:** Dekorator membuat endpoint; path parameter menunjuk, query parameter menyaring.
5	Request dan response model	**Inti:** Body divalidasi model Pydantic, ditolak 422 bila salah; response model menyaring keluaran.
6	Model dan integrasi basis data	**Inti:** Depends menyuntikkan sesi basis data ke setiap endpoint; tabelnya di P12.
7	Keamanan dan autentikasi	**Inti:** Hash password, token di header Authorization, diperiksa lewat dependency.
8	**API dan RESTful services**	**Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js**

Satu sumber daya, lima endpoint

ITERA

Metode dan path	Tindakan	Berhasil
`GET /matkul`	daftar, dengan query penyaring	200
`GET /matkul/{kode}`	satu mata kuliah	200, atau 404
`POST /matkul`	tambah dari body	201, atau 422
`PUT /matkul/{kode}`	ubah seluruhnya	200, atau 404
`DELETE /matkul/{kode}`	hapus	204, atau 404

Ingat

Path memakai kata benda jamak, bukan kata kerja: `POST /matkul`, bukan `/tambahMatkul`. Metode HTTP sudah menyatakan tindakannya (P1).

Memanggil dari Next.js

```
async function tambah(nilai) {
  const res = await fetch("http://localhost:8000/matkul", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(nilai),
  });
  if (res.status === 422) {
    const { detail } = await res.json();
    setGalat(detail[0].msg);
    return;
  }
  if (!res.ok) { setGalat("Gagal menyimpan.  Coba lagi.");
return; }
  setPesan("Mata kuliah tersimpan.");
}
```

Status lain di luar 2xx: pesan gagal. Berhasil: konfirmasi. Tiga keadaan antarmuka dari P1.

Tiga baris yang salah

Yang ditulis	Mengapa salah, dan perbaikannya
`return {"error": "tidak ada"}` untuk data tak ditemukan	Statusnya 200, sehingga frontend mengira berhasil. `raise HTTPException(404, ...)`.
`@app.get("/hapusMatkul/{kode}")`	GET tidak boleh mengubah data dan path memakai kata kerja. `@app.delete("/matkul/{kode}")`.
fetch dari Next.js gagal, Network menunjukkan 200	Diblokir CORS. Tambahkan asal `http://localhost:3000` di `CORSMiddleware`.

Pola kesalahannya

Ketiganya berasal dari kebiasaan lama: mengabaikan kode status dan metode HTTP dari P1.

Giliran kalian

ITERA

1 **Setengah jadi: hapus mata kuliah**

Lengkapi: `@app._____("/matkul/{kode}", status_code=_____)` dan di dalam fungsi, bila tidak ditemukan `raise HTTPException(status_code=_____)`.

2 **Mandiri: pencarian**

Tulis endpoint `GET /matkul/cari` dengan query `q` wajib minimal 2 huruf yang mengembalikan mata kuliah yang namanya mengandung `q`, tanpa membedakan huruf besar.

JAWABAN

Setengah jadi: delete, 204, 404. Mandiri: def cari(q: Annotated[str, Query(min_length=2)]): return [m for m in DATA if q.lower() in m[”nama”].lower()]; tulis sebelum /matkul/{kode} agar tidak tertangkap sebagai kode.

Hands-on bertingkat

ITERA

Tingkat	Waktu	Yang dilatih	Sub-CPMK
Tingkat 1, Dasar	10′	path operation, path parameter, query parameter, dan 404	11.2
Tingkat 2, Menengah	15′	model Pydantic, POST 201, PUT, DELETE 204, dan 422	11.1, 11.2
Tingkat 3, Tantangan	10′	CORS dan menyambungkan Next.js P7 ke API	11.2
Pengayaan	sisa	APIRouter dan dependency sederhana	11.1

Aturan

Berkas awal ada di `02 Hands-on/P11 - FastAPI - Hands-on/starter`. Kerjakan berurutan dari tingkat 1, simpan bukti tangkapan layar, dan sebutkan alasan setiap langkah.

Presentasi di papan

ITERA

Tiga mahasiswa diundi untuk menunjukkan hasil hands-on tingkat 2 atau 3 di layar, masing-masing sekitar 5 menit, lalu menjawab satu pertanyaan.

Kriteria	Sangat baik 75--100	Baik 65--74	Cukup 55--64	Kurang <55
Ketepatan hasil	berjalan dan sesuai soal	satu salah kecil	ada bagian salah	sebagian besar keliru
Alasan tiap langkah	setiap baris dijelaskan fungsinya	sebagian besar dijelaskan	penjelasan kabur	tanpa penjelasan
Menjawab pertanyaan	tanpa melihat catatan	dengan sedikit bantuan	menjawab sebagian	tidak bisa menjawab

Yang tidak maju tetap bisa menambah nilai lewat pertanyaan atau koreksi yang tepat.

Exit ticket Pertemuan 11

```python
@app.get("/mahasiswa/{nim}")
def detail(nim: str, aktif: bool = True):
    ...
```

1 Sub-CPMK 11.1

Untuk request `GET /mahasiswa/124140127?aktif=false`, sebutkan mana path parameter dan mana query parameter, beserta nilai dan tipenya di dalam fungsi.

2 Sub-CPMK 11.2

Tulis model Pydantic dan endpoint `POST /mahasiswa` yang menerima NIM 9 karakter dan IPK 0 sampai 4, membalas 201. Kode status apa yang kembali bila IPK diisi 4 + *A*?

3 Refleksi

Tulis satu kalimat: bagian mana yang paling membingungkan hari ini, dan mengapa?

Soal 1 dan 2 dinilai 0, 1, atau 2: benar dengan alasan, benar tanpa alasan atau salah kecil, belum benar.

Yang dibawa pulang

ITERA

	Bagian	Inti
1	Pengenalan FastAPI	FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	Instalasi dan konfigurasi	Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.
4	Routing	Dekorator membuat endpoint; path parameter menunjuk, query parameter menyaring.
5	Request dan response model	Body divalidasi model Pydantic, ditolak 422 bila salah; response model menyaring keluaran.
6	Model dan integrasi basis data	Depends menyuntikkan sesi basis data ke setiap endpoint; tabelnya di P12.
7	Keamanan dan autentikasi	Hash password, token di header Authorization, diperiksa lewat dependency.
8	API dan RESTful services	Kata benda di URL, tindakan di metode HTTP, kode status yang tepat.



Ada pertanyaan?

Sampai jumpa di Pertemuan 12: Basis Data PostgreSQL